

Research Progress Report 4

Computational Optimization of the Heisenberg Model

Sai Aung Naing Win | UC Davis Department of Physics | Supervisor: Prof. Rajiv R.P. Singh

September 07, 2026

1. Research Direction

I focused on improving the computational efficiency and organization of the Heisenberg model. The original implementation was enough for the 10x10 lattice calculations, but the computational cost quickly increased when increasing the lattice size, number of repeated trials, or number of parameter combinations being tested.

Since I will be continuing this research throughout the Fall Quarter, I wanted to improve the simulation framework before introducing more computationally demanding extensions of the model.

The main goal of the optimization is to reduce the runtime of the Monte Carlo simulations, make better use of the available CPU resources, and reorganize the code so that future experiments can be added and modified more easily. The two main performance improvements were parallelizing independent hysteresis trials across multiple CPU processes and using Numba compilation for the computationally intensive Monte Carlo functions which ultimately reduced the runtime by roughly 99%

2. Code Reorganization

Before optimizing the numerical calculations, I reorganized the simulation into separate Python files for the core physical functions, parallel processing utilities, configuration parameters, and individual experiments. Previously, most of the model, experiments, and plotting were contained within a single Jupyter Notebook.

The current structure separates reusable functions from the scripts used to perform specific experiments. Common parameters such as J , T , field range, field step, number of sweeps, and anisotropy strengths are also stored separately. This does not change the physical model, but it makes the simulation easier to extend.

3. Parallel Processing of Independent Trials

The first major optimization was to parallelize repeated hysteresis trials. In the previous implementation, the 20 trials for each condition were executed sequentially using a standard Python for loop.

I implemented parallel execution using Python's `ProcessPoolExecutor`, a generic parallel function that submits the same single trial function multiple times and distributes the jobs across a selected number of worker processes. Each worker completes an entire hysteresis calculation independently, while the magnetic field sweep and Monte Carlo updates within each individual trial remain sequential which avoids changing the update dynamics of the simulation.

The parallel calculations using 10 worker processes on the desktop CPU reduced the runtime for the full 7 condition, 20 trial data set from an estimated 74.2 minutes using the original serial implementation to approximately 14.2 minutes. This corresponds to a speedup of about 5.2x from multiprocessing alone.

4. Numba Compilation of the Monte Carlo Sweep

The next optimization targeted the main computational bottleneck. Most of the runtime is spent inside the Monte Carlo sweep, where a random lattice site is selected, a new Heisenberg spin direction is proposed, the energy difference is calculated, and the move is accepted or rejected according to the Metropolis rule. These operations are repeated a very large number of times during a hysteresis calculation.

I used Numba compilation on the most frequently executed numerical functions. Numba compiles these Python numerical loops into native machine code after the first call. This removes much of the Python overhead while preserving the same Monte Carlo update logic

The first compiled call takes longer because Numba must generate machine code for the functions and argument types. After compilation, subsequent calls reuse the compiled version and execute much faster. The resulting repeated calculations again produced discernible standard deviations and different coercive field values between trials.

5. Runtime Benchmark

To quantify the performance gain, I compared the same 10x10 hysteresis workload at several stages of optimization. The benchmark contains seven anisotropy conditions with 20 trials per condition, for a total of 140 hysteresis simulations. The simulations used $J = 1$, $T = 0.25$, $H = +4 \rightarrow -4 \rightarrow +4$, $dH = 0.2$ and 100 Monte Carlo sweeps at each field value.

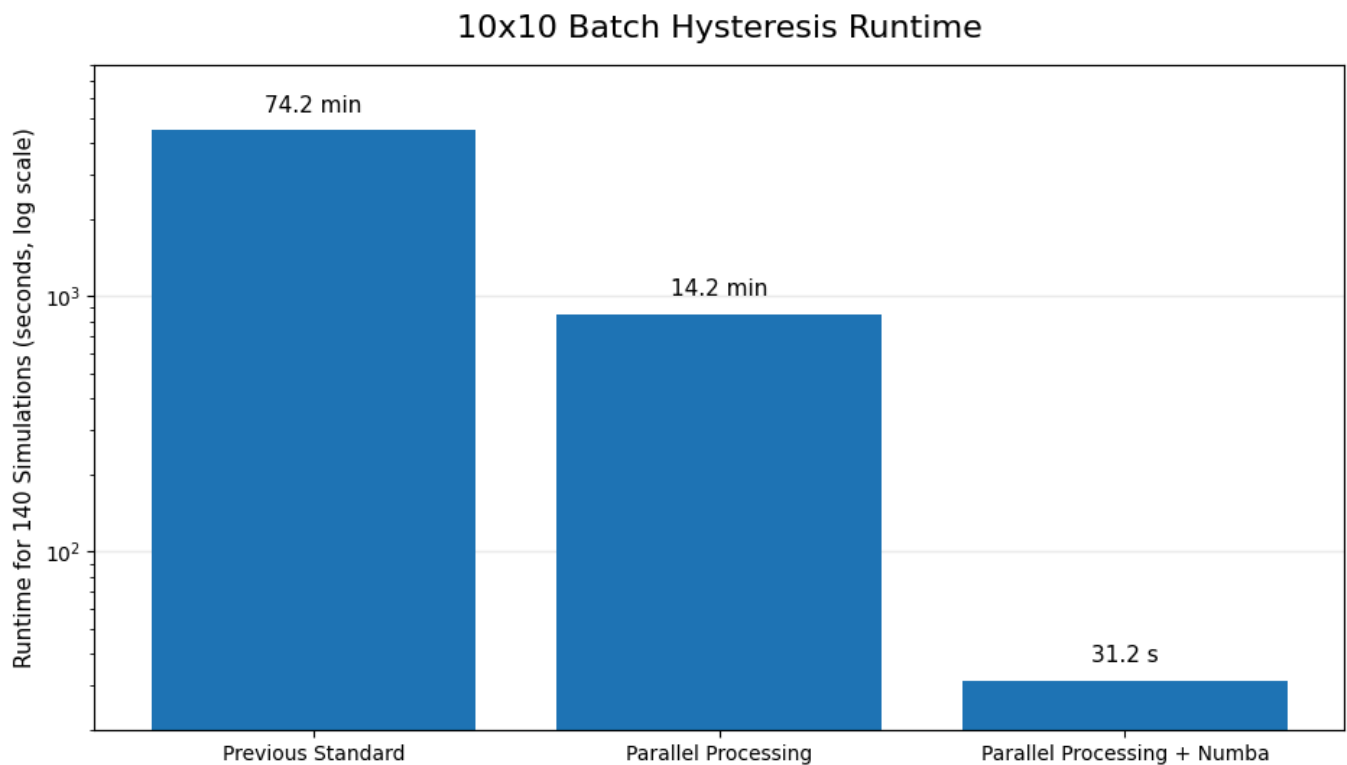


Figure 1: Runtime comparison for the 140 simulation 10x10 benchmark. Results shown here used 10 worker processes.

Compared with the multiprocessing only version, the addition of Numba reduced the 10x10 batch runtime from approximately 849 seconds to 31.2 seconds, a further speedup of about 27.2x. Relative to the estimated original runtime, the complete optimized workflow is about 143x faster for this benchmark.

6. Larger Lattice Test

The optimized simulation was then tested on a larger 50x50 and 100x100 lattice using the same 140 simulation batch. A 50x50 lattice contains 2,500 spins, 25 times the number of spins in the original 10x10 system, while a 100x100 lattice contains 10,000 spins, or 100 times as many spins as the 10x10 lattice.

The 50x50 batch completed in approximately 222 seconds, or 3.7 minutes

The 100×100 batch completed in approximately 817 seconds, or 13.6 minutes.

The runtime increased from approximately 222 seconds to 817 seconds, corresponding to an increase of about 3.7×. This is close to the expected linear scaling with the number of lattice sites showing that the optimized implementation continues to scale efficiently as the system size is increased.

Implementation	Lattice	Simulations	Runtime	Speedup
Previous Standard	10x10	140	4453s (74.2 min)	1.0x
Multiprocessing only	10x10	140	849.8s (14.2 min)	5.2x
Multiprocessing + Numba	10x10	140	31.2s	142.6x
Multiprocessing + Numba	50x50	140	222.1s (3.7 min)	—
Multiprocessing + Numba	100x100	140	816.9s (13.6 min)	—

Table 1: Benchmark of the simulation framework before and after multiprocessing and Numba optimization.

The larger lattice simulations also continued to show the same qualitative behavior observed previously. The coercive field decreases as the fraction of strongly anisotropic sites is reduced, and periodic distributions generally produce larger coercive fields than random distributions at intermediate concentrations. In the larger lattice, the spread of H_c values decreases, indicating reduced statistical variation across repeated simulations. This provides an additional check that the optimized code continues to reproduce the expected trends while allowing substantially larger systems to be studied.

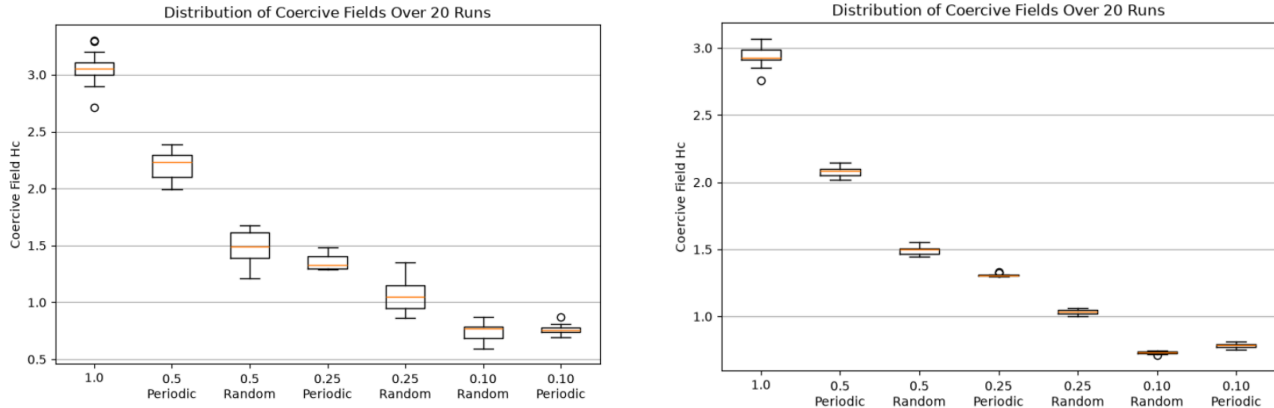


Figure 2: Comparison of 10×10 (left) and 50×50 (right) lattice results.

7. Future Simulations

The main purpose of this optimization is not only to reduce the runtime of the current experiments, but also to make more computationally demanding extensions more feasible. Larger lattices also reduce the influence of very small system effects and make it possible to represent spatial structures such as grains, interfaces, clusters, and controlled separations between anisotropic regions more realistically.

For example, future simulations could compare several lattice sizes, different magnetic moments, grain sizes, interface exchange values, or anisotropy distributions while still repeating each condition enough times to estimate the mean behavior and statistical variation.